Toasty: Speeding up network I/O with cache-warm buffers

Abstract

Modern NICs DMA packets directly to the LLC using technologies like DDIO, reducing access latencies for networking applications. Prior work has observed several performance issues with DDIO when the working set size of the packet buffers does not fit into the LLC. For example, the *leaky DMA* problem arises when incoming packets evict older packets that have not yet been processed by the application from LLC, causing them to be fetched again from main memory. While using a smaller pool of packet buffers that fits in cache is an obvious solution, this may result in the NIC running out of buffers to DMA packets into when a burst of packets arrives in a small interval. This paper proposes Toasty, a system that mitigates this tradeoff between high throughput and resilience to packet loss that arises when sizing the network packet buffer pool. While prior work has proposed solutions that change the NIC hardware to address this problem, Toasty is a software-only solution that can be deployed on commodity NIC hardware. Toasty manages the packet buffer pool as a LIFO stack instead of a FIFO queue, and adapts the number of buffers populated into the NIC hardware RX ring based on incoming packet load and application processing rate. Together, these changes enable Toasty to recirculate a small working set of cache-warm buffers in steady state, while falling back to a larger pool of buffers during traffic bursts. We implement Toasty over the AF_XDP kernel bypass framework, and our evaluation shows that Toasty improves network throughput for a variety of network functions by up to 78% over the default buffer pool implementation of AF_XDP, and achieves almost zero cache miss rates. We also show that Toasty matches the performance that can be achieved with a small buffer pool that fits in cache, while being much more resilient to bursty traffic.

1 Introduction

Innovation in chip manufacturing has led to an explosion in the speeds at which computer networking is possible today. However, CPU frequencies have not progressed at the same pace, forcing networking software to play catch-up in the ability to process packets over high-bandwidth links efficiently [1, 5, 6, 17, 22–24, 29]. Recently, kernel bypass frameworks like DPDK [10] and AF_XDP [3, 15] deliver significant improvements in software packet processing rates by removing the kernel bottleneck from the network I/O data path. To avoid dynamic buffer allocation and packet copy overheads, these frameworks pre-allocate a large pool of packet buffers in userspace memory, and the Network Interface Card (NIC) receives pointers to these free buffers from the device driver. When packets arrive over the network, the NIC DMAs incoming packets into the free buffers, which are directly processed by the application in a zero-copy manner, and then recycled back into the free pool.

In addition to software frameworks, Direct Cache Access (DCA) is a hardware technology to help applications process packets faster, with Intel's DDIO [11] leading the charge. With the limited time budget available to process packets with high-speed NICs, fetching every packet from memory is no longer viable [7, 14, 18]. For this reason, DDIO enables the NICs to DMA packets directly into the last-level cache (LLC). This reduces the access latencies of packets and memory bandwidth usage for applications, and hence improves network I/O performance. DDIO uses a limited number of LLC ways (10% of LLC ways), to prevent network I/O traffic from interfering with co-located applications.

Previous work [1, 5, 17, 23, 24, 30, 32, 34] has identified several shortcomings with DDIO. If the size of the packet buffer pool is larger than what can fit into the DDIO ways of LLC, then network buffers contend for the limited available space, and several problems result from this cache contention. For example, if packets arrive faster than the application can process them, newer packets may evict older packets from the LLC even before the application has processed them. This phenomenon is popularly known as the *leaky DMA problem* [30], and causes the application to fetch most of the packets from memory even with DDIO in place. A more subtle problem arises in the case of packet buffers that have been processed by the application and added back to the free buffer pool. These free buffers continue to stay in various levels of caches, with the data never being accessed again (because the application is "done" with the packet), until they are either evicted due to cache contention, or the NIC reuses the buffers to DMA new packets. If free network buffers are evicted from the cache before reuse by the NIC, their contents will be written back to memory, because the cachelines would be dirty. However, such writebacks are wasteful because the packet content will be overwritten when the packet buffer is reused for DMA in the future. This is called the *unnecessary writebacks problem* [1, 31], and there is no way for hardware to identify and avoid these writebacks.

The above problems seem to indicate that one must use a small pool of free packet buffers, which fits snugly into LLC DDIO ways, as proposed in prior work [17, 23, 24, 30]. A small pool of buffers will significantly ameliorate the leaky DMA problem, because packet buffers will not have to contend for LLC space and evict other packet buffers, thereby improving cache hit rates and application throughputs. A small

pool of packet buffers also ensures that processed buffers that are added back to the free pool are reused quickly again for DMA before being evicted from the cache. Such reuse of cache-warm buffers for packet DMA avoids wasteful write-back traffic to memory, because when the NIC tries to DMA a packet into a buffer that is already present in the CPU cache, the buffer is updated in place without accessing memory. Our experiments with different types of network functions over the AF_XDP kernel bypass framework (§2) show that most network functions achieve optimal application (and cache) performance with a small pool of buffers that fit in the DDIO LLC ways. However, under highly variable or bursty traffic conditions, we also see that a small buffer pool leads to packet drops due to the exhaustion of packet buffers available to the NIC for DMA, a phenomenon observed in prior research as well [1, 6, 17, 23, 24].

The goal of our work is to mitigate this tradeoff between high network I/O throughput and resilience to packet loss that one encounters when sizing the packet buffer pool. Prior work has proposed solutions that either redesign the NIC-CPU interface, or leverage advanced hardware features available in modern smart NICs, to better utilize the LLC space available for network packets [17, 22–24, 28]. Our work seeks to explore a solution that works on today's commodity NICs, for immediate deployment and impact.

To this end, this paper introduces Toasty, a software framework that enables the reuse of a small working set of cache-warm packet buffers in steady traffic conditions, while automatically adapting to a larger buffer pool to avoid packet drops due to traffic bursts. Our work consists of two key ideas. First, Toasty manages the pool of free packet buffers as a Last-In-First-Out (LIFO) stack, rather than a First-In-First-Out (FIFO) queue as is commonly done. This ensures that network packet buffers that have been recently processed and freed up by an application, and hence likely to be warm (and toasty!) in caches, are reused for the DMA of new packets immediately. However, using a LIFO stack for the memory pool alone is not enough, because the hardware RX ring (in which the NIC stores pointers to the free buffers) still works as a queue, and changing its behavior from FIFO to LIFO requires hardware changes. Therefore, the second key idea of our work is to adaptively control the number of packet descriptors that are filled in the hardware RX ring. Under steady incoming load, the device driver refills the RX ring with a small number of packet descriptors from the head of the free pool, which, given our LIFO buffer pool design, are likely to be warm in the cache. However, when the device driver detects a high traffic rate that causes a rapid depletion of free buffers available in the RX ring, it populates the NIC RX ring with a larger number of buffers, temporarily pulling in cold packet buffers from beyond the warm region at the head of the buffer pool. This adaptive filling of the hardware RX ring by the device driver ensures that we reuse only cache-warm buffers in steady state, while effectively handling traffic volatility. Toasty implements these ideas on top of the AF_XDP kernel bypass framework, though our ideas can be applied to other packet I/O frameworks as well.

Our evaluation of Toasty with a variety of network functions shows that Toasty improves network function throughput by up to 78% over the default AF_XDP implementation, when both systems use a larger buffer pool that is resilient to varying traffic conditions. Our gains are even more compelling at up to 86% when there is increased contention in the LLC from co-located applications. Further, the performance of Toasty with a larger buffer pool matches the performance of the default AF_XDP with an ideal throughput-maximizing small buffer pool, while being a lot more resilient to bursty traffic conditions. We show that our performance gains come from improved CPU cache hit rates (due to better reuse of cache-warm buffers) and reduced writeback traffic to memory. Our evaluation also demonstrates the synergy between the two key components of Toasty—the performance gains of Toasty are higher than what can be achieved with userspace-only changes (i.e., LIFO memory pool without adaptive sizing of the RX ring) that have been proposed for frameworks like DPDK [25], or from adaptive RX ring resizing alone.

The rest of the paper is organized as follows. We begin with some background and motivation (§2), then discuss the design and implementation of Toasty (§3), followed by our evaluation (§4), related work (§5), and conclusions (§6).

2 Background and Motivation

2.1 Packet Buffer Lifecycle

We first review the lifecycle of a packet buffer in popular high-performance kernel bypass frameworks like AF_XDP [3, 15] and DPDK [10]. While DPDK uses a userspace poll-mode device driver, AF_XDP redirects packets from a hook in the kernel device driver. This minimal kernel involvement adds a small overhead in AF_XDP as compared to DPDK, but also provides several advantages, e.g., interrupt-based processing for lower idle CPU usage and the ability to easily share the NIC across applications. For these reasons, Toasty builds upon the AF_XDP framework, although our ideas can also easily extend to other frameworks.

To avoid dynamic memory allocation and packet copy overheads, applications built over such frameworks use a contiguous memory region of pre-allocated packet buffers to store network packet data. For example, an AF_XDP application initializes a userspace buffer pool called UMEM, where each packet buffer is typically 4KB in size. The application and the device driver exchange packet "descriptors", i.e., pointers to buffers in this region, using several *queues*. In AF_XDP, every UMEM is linked with four queues called the Fill Queue (FQ), RX Queue (RXQ), TX Queue (TXQ), and Completion Queue (CQ), as described in [3, 19]. Further, the device driver and the NIC hardware share one or more RX

and TX *rings* filled with packet descriptors, that the hardware accesses when receiving or transmitting packets.

Figure 1 illustrates the lifecycle of a packet buffer in the AF_XDP framework, focusing on the RX path (see [3, 19] for a detailed description). Initially, 1 the application populates the FQ with "free" buffers. 2 The kernel driver then moves some of these descriptors to the NIC's RX ring. When a network packet arrives, 3a the NIC dequeues a descriptor from the RX ring, and performs a DMA of the packet into the free buffer. If DDIO is enabled, the corresponding cache lines are allocated (or updated) in the DDIO LLC ways. The driver moves these buffer pointers into the RXQ 3b, and notifies the AF_XDP application to start processing these packets. The application dequeues 4a and processes the packets from the RXQ 4b. If the packet is consumed locally, the application marks the buffer containing it as "free" 5a, and the free buffer descriptors are recycled into the FQ for reuse immediately 5b. If a packet needs to be forwarded back into the network, its buffer can only be marked as "free" after the network device completes the transmission via the transmit path. Note that only buffer pointers move between these various queues; there is no data copying.

AF_XDP supports two main operating modes: interrupt mode and busy poll mode. In the interrupt mode, the device driver is scheduled to run on a core upon an NIC interrupt on that core, and it performs two critical tasks when it executes. First, it cleans DMA'd packets from the NIC RX ring and moves them to the AF_XDP RXQ for application processing (step 3b in Figure 1). Second, it "consumes" free buffers from the AF_XDP FQ, placing them in the NIC RX ring for them to be used for DMA of future incoming packets (step 2 in Figure 1). The execution of the kernel driver and application threads is interleaved via context switches. In contrast, with the busy poll mode of AF_XDP, the kernel driver logic executes when the application invokes system calls to poll and read data from the NIC itself, and there are no context switches between application and driver threads. While the interrupt mode achieves low idle CPU usage, the busy poll mode is more effective at high loads as it avoids context switch overheads [19, 20]. For this reason, we implement and evaluate all applications in the busy poll mode in this paper, though our ideas extend to other modes as well.

2.2 Challenges in sizing packet buffer pool

The working set of packet buffers of an application depends on several factors (the size of the packet buffer pool, the RX ring size, the rate at which the application can process and recycle buffers, and so on), and is at most the size of the packet buffer pool. We now show some empirical results to highlight the difficulties in sizing the pool of packet buffers, especially when DDIO is turned on. While one line of thought is to limit the pool of buffers to fit within the LLC for optimal LLC usage [30], the counterpoint is that the NIC may run

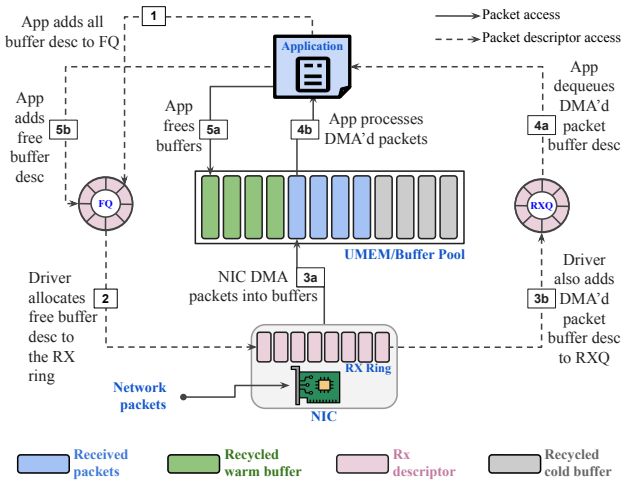


Figure 1. Lifecycle of a packet buffer

out of empty buffers in case a burst of packets arrives [6, 17, 23, 24]. We will now explore this tradeoff empirically.

For this exploration, and for experiments in the rest of the paper, we use six popular applications/network functions that we implement (or port) over AF_XDP: (i) **NAT** looks up and rewrites IP address bytes in the packet headers after reading them. (ii) **IDS** is an application that reads and compares bytes in the payload with a pre-defined pattern. (iii) **Decryption** is a proxy server in which the application reads the entire payload, decrypts it, and rewrites the contents of the packet. (iv) **L2Fwd** is a swap-and-forward application which reads and updates the MAC address, and then forwards the packets back into the network. (v) **MICA** is an optimized key-value server that handles get/put requests in the packet, retrieves/updates key-value pairs in memory, then sends back the packet to the client [16]. (vi) **Maglev** is a hashing-based L4 load balancer [4].

We run all of these applications on an Intel(R) Xeon(R) Gold 5418Y CPU, receiving packets over an Intel 100G E810 NIC, generated by the PktGen [33] load generator running on a different machine. We run each application on a single core, with a single RX ring capable of holding 4096 descriptors in the NIC bound to the application core. We measure saturation throughput (in Gbps) for varying buffer pool sizes, ranging from 256 (extremely low) to 16384 (moderately high) packet buffers. The L3 cache on the system is 45 MB in size and is 15-way set associative, with DDIO utilizing the last two ways. The packet size in these experiments was 512 bytes, while the packet buffer was 4KB. Hence, the L3 cache (DDIO ways) are capable of holding a minimum[1] of 1536 network packet buffers simultaneously.

Figure 2 shows single-core saturation throughput for all applications described above, as we vary the number of

[1]We use the term "minimum" because DMA can also happen in other LLC ways as well if the packet buffer is already present in LLC in some other way, which is referred to as DMA bloating [1].

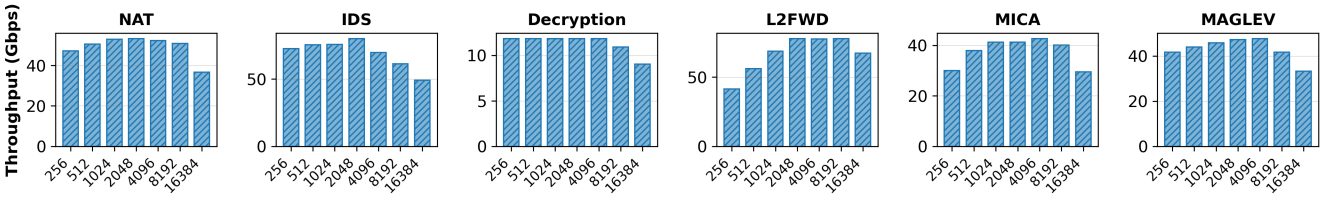


Figure 2. Application throughput vs. size of the packet buffer pool

Modes	L2 Miss % (LLC Miss %)					
	NAT	IDS	Dec.	L2Fwd	MICA	Mag.
Default AF_XDP	37.3 (0.85)	64.77 (8.61)	61 (8.88)	39.9 (0.58)	39.5 (4.21)	37.8 (0.94)
Ideal AF_XDP	15.4 (0.004)	55.08 (0.018)	52.7 (0.023)	15.4 (0.003)	26.2 (0.075)	15.4 (0.045)

Table 1. L2 and LLC cache miss rates for all applications

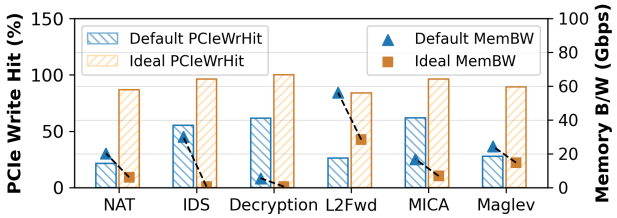


Figure 3. PCIe write hit rate and memory bandwidth usage

buffers in the packet buffer pool. The results are averaged over 5 runs of the experiment, each of which lasted for 300 seconds. We see from the figure that applications do not perform well in the extreme cases of using a very low number of buffers (256) or a moderately high number of buffers (16384). The decryption workload is the only exception to this rule, as it is compute-bound and is able to saturate a core at very low packet rates. For each application, we pick the best configuration of the buffer pool size and RX ring size that gives the highest throughput, and call it the "ideal" AF_XDP configuration (for that application) in the rest of the paper. We will refer to the common configuration of AF_XDP with a comfortably large buffer pool (16384 buffers) and RX ring (4096) as the "default" AF_XDP configuration.

Table 1 shows the L2 and LLC miss percentages, computed using the perf tool [8], for the various applications, using the "default" and "ideal" buffer pool size configurations described above. We see that applications have significantly higher L2 and LLC miss rates for the default configuration with a larger packet buffer pool, as compared to the ideal configuration where the packet buffers fit in LLC. [2] Figure 3 shows the

[2]The reader may wonder why we measure L2 performance if DDIO only works at the LLC. We find that due to hardware prefetching (that is on by default), DDIO also improves L2 performance, because packets are prefetched into L2 before access. We find L2 miss rates to be significantly higher and not impacted by DDIO when prefetching is turned off, and we see a larger impact at LLC itself in such cases.

Modes	Microburst Packet Drop %					
	NAT	IDS	Decrypt	L2Fwd	MICA	Maglev
Ideal	7.843	0	14.835	0.002	1.571	11.884
Default	0.001	0	0.002	0.001	1.628	0.001

Table 2. Packet drops (in %) with microbursts

Modes	Poisson Traffic Packet Drop %					
	NAT	IDS	Decrypt	L2Fwd	MICA	Maglev
Ideal	3.382	1.831	5.756	0.054	4.126	6.066
Default	1.22	0.877	1.475	0.032	2.76	1.367

Table 3. Packet drops (in %) with Poisson traffic

PCIe hit rates and write memory bandwidth usage for the same experiments. The former signifies the number of times the NIC found the buffer already in the LLC in the event of a DMA (and hence did not have to write-allocate cache lines in the DDIO ways, but could simply update them) whereas the latter is the amount of writeback traffic generated towards main memory, e.g., in the event of a PCIe write miss. These results show lower PCIe hit rates and higher memory bandwidth usage, respectively, for the default AF_XDP configuration, in comparison to the ideal configuration, and are consistent with the leaky DMA and unnecessary writeback problems that occur with a larger buffer pool.

So, why then do real-life applications need to use larger buffer pools? The answer lies in the ability to handle diverse traffic conditions. In all experiments so far, we generated uniform traffic at a specified rate, which was varied across experiments to saturate the application. We now change the load generator to generate bursty traffic in two different ways. First, we generate traffic with "microbursts", where we send a pulse of 2048 512-byte packets every 1ms, with no other traffic for the remainder of the 1ms period, such that the overall average rate is below the application's processing capacity. Note that this burst size is still well below the total RX ring size in the NIC (4096 buffers in our case), and is similar to traffic patterns that can occur in real-life situations, e.g., during TCP's bandwidth probing. Next, we generate variable traffic following the Poisson distribution, where the rate of the incoming traffic varies every 5ms according to a Poisson distribution, whose mean is equal to the packet processing capacity of the application. Tables 2 and 3 show the packet drop percentage (dropped at the NIC) for all network functions under these variable traffic conditions. We see from these results that the "ideal" configuration, which

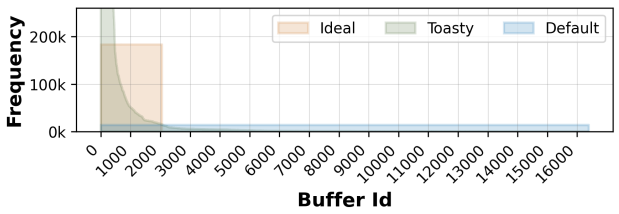


Figure 4. Frequency of buffer use for packet DMA

resulted in the highest throughput for applications under a steady packet arrival rate, suffers from a non-trivial packet loss when traffic arrives in bursts, because the NIC temporarily runs out of packet buffers to DMA packets into. On the other hand, the default configuration with a lower saturation throughput is quite resilient to bursty traffic conditions. Note that the source of variability that leads to such packet loss can also come from the application, e.g., a temporary increase in processing delay in the application can cause a smaller packet buffer pool to run dry, leading to packet loss.

The experiments above illustrate the tradeoff between the requirements of high throughput and resilience to packet loss under traffic bursts. This problem is exacerbated for multi-core applications that receive packets from multiple NIC RX rings, where every core must have enough buffers in its RX ring to avoid packet loss, leading to an even higher working set size [23]. While solutions have been proposed to this problem in prior work [17, 23, 24, 28], they either redesign the hardware NIC-CPU interface, or leverage advanced features of modern smart NICs, while our goal is to come up with a solution that works on commodity NIC hardware. A simple strawman approach to this problem is to dynamically resize the packet buffer pool or RX ring size in response to incoming traffic patterns or application processing rates. However, this is not feasible at run time, as these operations require a restart of the device and/or application.

The key contribution of our work is to come up with a practical solution that allows us to reuse a small set of cache-warm packet buffers under suitable traffic conditions, thereby avoiding issues like leaky DMA, while also being able to transition to a larger pool of buffers dynamically when packet losses are imminent. Figure 4 shows the frequency of use of the 16384 buffers in the buffer pool for DMA, when running the NAT application with Toasty as well as with AF_XDP configured with the default and ideal buffer pool sizes. We see that the ideal and default AF_XDP configurations use all buffers uniformly, while Toasty reuses the first thousand or so buffers more frequently under low load, but pulls in the other buffers to handle traffic bursts, thereby achieving both high cache reuse and negligible packet drops.

3 Toasty Design and Implementation

This section describes the design and implementation of Toasty. Our system consists of two components built over the AF_XDP packet I/O framework. First, we modify the userspace packet buffer pool management in AF_XDP to work as a LIFO stack rather than a FIFO queue, so that recently recycled warm buffers are reused for future packets immediately (§3.1). Next, we modify the kernel device driver that populates the NIC RX ring to adapt the number of buffers in circulation based on incoming load and application processing rate, in a way that improves cache locality while being resilient to packet loss (§3.2).

3.1 LIFO buffer pool in userspace

The fill queue (FQ) holds pointers to the free packet buffers from the userspace packet buffer pool in AF_XDP, and the size of the FQ is typically the number of buffers in the buffer pool. To implement the userspace component of Toasty, we change the FQ implementation in the AF_XDP userspace library to add newly replenished packet buffers to the head of the FQ instead of at the tail, thereby allowing for quicker reuse of the cache-warm free buffers.

The FQ is a single-producer single-consumer circular queue, where the AF_XDP application is the producer (enqueues buffer pointers in the FQ), and the AF_XDP kernel driver is the consumer (dequeues buffer pointers from the FQ). The synchronization between the AF_XDP application and the driver is managed via two pointers, the *producer* pointer and the *consumer* pointer. As shown in Figure 5(a), the default implementation of the AF_XDP userspace library places recycled free buffers at the *producer* pointer in the FQ, and the driver dequeues buffers from the *consumer* pointer to allocate to the NIC RX ring. Because the FQ follows FIFO semantics by default, the free buffers being enqueued by the AF_XDP application at the producer pointer will be consumed by the AF_XDP driver only after it has cycled through all the other free buffers present in the buffer pool, by which time these recently recycled (and likely cache-warm) buffers may even be evicted from the LLC.

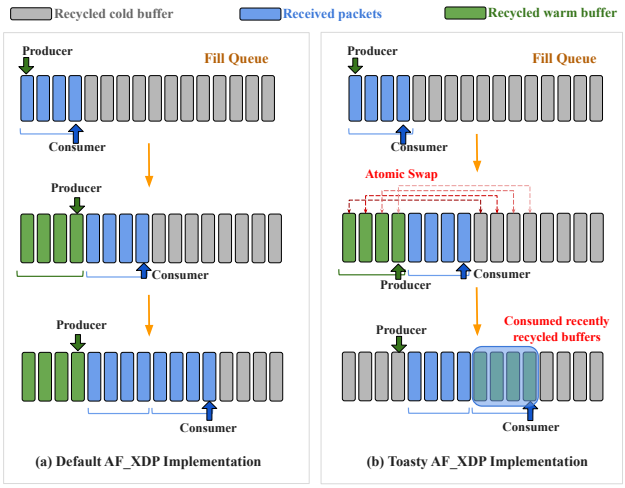


Figure 5. Toasty userspace changes in AF_XDP

```
def produce(buf[], *producer, *consumer):
  for i, buffer in enumerate(buf):
    # fill buffer into FQ
    *(producer + i) = buffer
    # critical addition in Toasty
    ATOMIC_EXCHANGE(*(consumer + i), *(producer + i))
  producer += len(buf)

def consume(buf[], *consumer):
  for i, buffer in enumerate(buf):
    allocate_in_rx_ring(*(consumer + i))
  consumer += len(buf)
```

Listing 1. Toasty atomic swap

Figure 5(b) shows the modification we make to this logic in Toasty to improve cache reuse. After the application processes and recycles a batch of packets at the *producer* pointer, we atomically swap these packet descriptors with those present at the *consumer* pointer. This exchange essentially moves the buffers that were just recycled by the application to the front of the queue, while moving the buffers that were originally present at the front to the back of the queue. The *produce* function in Listing 1 shows the pseudocode of the logic implemented in the AF_XDP userspace library in Toasty. Note that we do not make any changes to the consumer logic shown in the *consume* function. Now, with this change in place, the most recently processed buffers will be quickly reused for DMA of incoming packets. Because these buffers are likely to be cache-warm (present in LLC or even in the lower-level caches), a DDIO-capable NIC will most probably encounter a PCI write hit during DMA of a packet into the LLC and will update the buffer in place. This will lead to fewer evictions from LLC, mitigating issues like the leaky DMA problem or the wasted writeback problem. It is important to note here that Toasty only swaps buffer pointers inside the FQ, and not the actual buffer contents themselves. For the atomic exchange of buffer pointers, we use GCC's atomic built-in function *__atomic_exchange_n()* to swap the packet descriptors present at the producer pointer. We quantify the overheads of this swap in §4.

An astute reader may notice the possibility of race conditions between the producer and consumer functions shown above. What if the consumer runs concurrently with the producer? Is it possible that the kernel driver may consume and move a buffer from the head of the FQ to the hardware RX ring, while the producer is swapping this buffer with a newly recycled warm buffer? This may result in the buffer being incorrectly handled, e.g., the NIC may DMA a packet into this buffer, and the application may never process it (believing it to be a free buffer). We would like to clarify that such race conditions do not occur in the busy poll mode of AF_XDP that is used in Toasty, because the kernel driver's consumer code runs in the context of the userspace application itself, when the application invokes a system call to poll and read from the device. Therefore, the producer and consumer functions cannot run concurrently, and hence there is no scope for such race conditions in Toasty.

However, the above described race conditions are possible in the interrupt mode of AF_XDP, if the application and kernel driver threads execute concurrently, either in parallel on different cores, or interleaved via context switching on the same core. In such cases, we will need to set up more synchronization between the producer and consumer functions shown above. For example, we can remember the ring entry at the head of the queue, and abort the swap if we notice that this entry has changed, say, using the compare-and-swap atomic operation. Such synchronization involves a higher overhead, but may be acceptable in low-load scenarios that are typically suitable for the interrupt mode of AF_XDP.

3.2 Adaptive RX refill in kernel driver

The default behavior of NIC device drivers is to keep the RX ring fully stocked with packet buffers, so that the NIC never needs to drop incoming packets due to the unavailability of free buffers to DMA packets into. Now, note that the hardware RX ring is still a FIFO queue and cannot become a LIFO stack barring hardware changes. Therefore, the cache-warm buffers pulled from the head of the packet buffer pool (after Toasty userspace modifications) will be used to DMA incoming packets only after all the other buffers in the ring ahead of them have been used up. Therefore, the userspace changes of Toasty (and similar ideas proposed in the context of userspace kernel bypass drivers, like DPDK [25]) alone may not be fully effective in improving cache reuse of warm buffers, unless we limit the RX ring size to small values [27]. However, limiting the size of the NIC RX ring may result in the driver being unable to replenish the ring quickly enough during traffic bursts, which can lead to packet loss. Therefore, the kernel device driver in the AF_XDP framework is modified in Toasty, to dynamically adapt the number of packet descriptors being refilled into the NIC RX ring from the userspace packet buffer pool (FQ), in order to maximize the reuse of cache-warm buffers, while also avoiding packet drops due to the NIC ring running dry.

A simple method to estimate the optimal number of buffers in the ring could be to read NIC registers that keep track of packet drops and increase the number of buffers to the point where there are no packet drops. However, reading NIC registers frequently incurs a high performance overhead; also, the NIC registers are not updated in real time. Instead, Toasty uses driver-level metrics that are readily available in software to estimate the optimal number of buffers to fill in the RX ring. This estimate is made in every iteration when the driver code is invoked to replenish the RX ring, e.g., every time a batch of packets is processed by the kernel's bottom-half interrupt handler, typically every few milliseconds. Note that we do not reconfigure the size of the NIC RX ring itself to this estimated value in every iteration, because doing so would require a restart of the device and application, and cannot be done fast enough to react to changes in incoming packet rate. Instead, we configure a reasonably large size for

the RX ring, but populate it only with as many buffers as required to comfortably handle the current traffic load.

Pseudocode 1 Adaptive Refill of NIC RX Ring

```
1:  N_FQ ← Number of packets recycled by app. in previous iteration
2:  N_RXQ ← Number of packets DMA'd by the NIC in current iteration
3:  N_avail ← Number of buffers available in RX ring for DMA
4:  k ← Tunable threshold to maintain a reserve of buffers in RX ring
5:
6:  if N_avail < k * N_RXQ then
7:      if N_RXQ < 50% of link capacity then
8:          Fill min(k * N_RXQ, N_FQ) buffers
9:      else
10:         Fill k * N_RXQ − N_avail buffers
11: if N_avail >= k * N_RXQ then
12:     if N_RXQ < 50% of link capacity then
13:         Pass
14:     else
15:         Fill min(k * N_RXQ, N_FQ) buffers
```

The algorithm used by the kernel driver in Toasty to estimate the number of buffers to fill in the RX ring is shown in pseudocode 1. The inputs on which the algorithm bases its decision are shown in lines 1–3. At the start of every iteration, the algorithm computes N_{FQ} (the number of buffers recycled into the FQ by the application since the previous iteration), N_{RXQ} (the number of packets that have been DMA'd by the NIC since the last iteration, and moved to the RXQ of the AF_XDP application), and N_{avail} (the total number of free buffers available in the NIC RX ring for use for DMA in the future). Note that these metrics are readily available from the state maintained by the driver, and from the values of the various head and tail pointers of the software queues. The algorithm then refills the RX ring with cache-warm buffers as far as possible, while trying to ensure that the number of buffers available for DMA in this iteration is at least k times the number of buffers used up for DMA in the last iteration (N_{RXQ}), where k is a configurable parameter (line 4). Our current implementation uses $k = 10$, and we evaluate our sensitivity to this parameter choice in §4.

If the number of available buffers is below this threshold (lines 6–10), the driver tries to place more buffers into the RX ring to mitigate this shortfall. However, if the number of packets received in the previous iteration indicates that the link bandwidth utilization is under 50% (any reasonable threshold will work here, as we show in §4), then we surmise that there is no imminent danger of the NIC running out of free buffers for DMA, so we refill the ring with only the recently recycled N_{FQ} warm buffers, subject to the maximum limit of $k * N_{RXQ}$. That is, when the rate of packet arrival is low enough that there is no danger of packet drops, we fill the NIC ring mostly with cache-warm buffers that have been recently recycled by the application. However, if the rate of packet arrival indicates that the link utilization is high, then we refill the ring to bring the number of buffers to our desired value of $k * N_{RXQ}$, pulling in cache-warm as well as cache-cold buffers from the packet buffer pool. This ensures that we are able to adapt the number of buffers in the ring to effectively handle a burst.

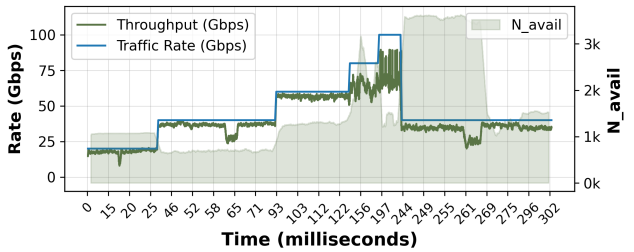


Figure 6. Toasty buffer adjustment with different traffic rates

Next, if the number of free buffers N_{avail} is above the desired limit $k * N_{RXQ}$ (lines 11–15), and the rate of arrival of packets is also below 50% utilization, then we assume that there is no danger of the NIC running out of buffers any time soon, and hence do not refill any new buffers into the ring in this iteration. This helps us drain out the excess buffers in the ring that were possibly brought into circulation to handle a previous burst, thereby allowing the working set size to settle to a lower value after a traffic burst ends. However, if we find the link utilization to be high, we still bring in the recently recycled N_{FQ} buffers, subject to the maximum limit of $k * N_{RXQ}$, into the RX ring. This prepares us to handle the ongoing high traffic load, but the high enough reserve of buffers already present in the ring means that we can afford to refill only cache-warm buffers into the ring. Note that in all of the above cases, we always restrict the total number of buffers to the maximum of the RX ring capacity.

Figure 6 illustrates how the Toasty driver dynamically allocates RX ring buffers. We run the L2Fwd application on AF_XDP under varying input traffic rates (20 to 100 Gbps), and the figure shows the incoming traffic rate, the application throughput with Toasty, and the number of buffers available in the NIC RX ring as a function of time. Note that the maximum application throughput in this case is around 78 Gbps, limited by CPU processing of the application. We can see from the figure that Toasty adapts the number of available buffers in the NIC to match the incoming load. After an initial calibration period, Toasty maintains a low number of buffers in the NIC, and all of these would likely be cache-warm due to the small working set size. However, as the traffic increases over a few iterations, Toasty increases the number of buffers as well, to adapt to the increasing rates of depletion of buffers in the RX ring. This avoids packet losses while potentially using a larger set of cache-warm and cache-cold buffers. Finally, after the traffic burst ends, Toasty adapts back to a low buffer level within a few tens of milliseconds again. In contrast, a static configuration of the number of buffers in the RX ring, either to a high value (in the default configuration) or to a low value (in the ideal configuration with the best throughput) attains a lower throughput, either

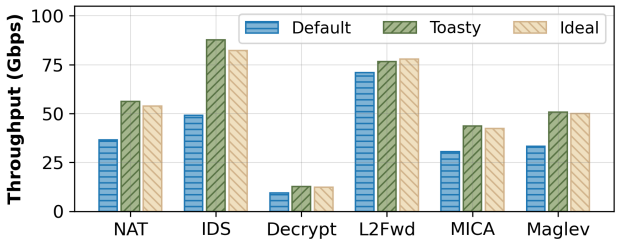


Figure 7. Single core throughput: Toasty vs. baselines

Toasty	L2 and LLC Miss %					
	NAT	IDS	Decrypt	L2Fwd	MICA	Maglev
L2	15.98	24.44	33.02	14.66	26.20	16.50
LLC	0.008	0.045	0.335	0.007	0.075	0.010

Table 4. Toasty L2 and LLC cache miss rates

Modes	IPC					
	NAT	IDS	Decrypt	L2Fwd	MICA	Maglev
Default	2.90	2.04	2.36	2.67	2.15	2.96
Ideal	3.68	2.65	2.36	2.98	2.69	3.89
Toasty	3.70	2.58	2.49	2.75	2.32	3.86

Table 5. IPC: Toasty vs. baselines

Modes	Latency (in microseconds)		
	Mean	99 Percentile	99.99 Percentile
Default	22.03	37.31	93.49
Ideal	15.88	25.17	38.8
Toasty	12.51	27.31	34.98

Table 6. Latency of L2Fwd: Toasty vs. baselines

due to the leaky DMA problem (caused by a large working set size) or from packet drops (caused by insufficient buffers in the RX ring), as we show in §4.

Finally, in addition to changing the RX ring refill logic, Toasty also modifies the transmit path of the kernel to proactively transmit packets in the TXQ. This change expedites the return of transmitted packet descriptors back into the free buffer pool and ensures that cache-warm buffers get recycled quickly. This change may hurt performance due to reduced TX batch size during transmissions. However, we see in §4 that the benefits of our change outweigh the costs, leading to overall improved performance.

4 Evaluation

We now evaluate the effectiveness of Toasty through extensive experimentation. We first compare the performance of Toasty with several baseline AF_XDP configurations (§4.1), e.g., the "default" AF_XDP configuration (large buffer pool) and the "ideal" configuration (smaller buffer pool size that maximizes throughput, similar to prior work like ResQ [30]). Next, we evaluate Toasty under varying traffic conditions (§4.2), co-located workloads (§4.3), and with a varying number of application cores and packet sizes (§4.4). In addition to the default AF_XDP baseline, we also compare Toasty to several systems in related work, i.e., ShRing [23], LIFO memory pool in DPDK [25], and software prefetching (§4.5). Finally, we report several microbenchmarks, ablation, and parameter sensitivity studies on Toasty (§4.6).

Experimental Setup. The experimental setup is similar to the one described for the experiments done in §2. The server machine running the NFs built over AF_XDP has Ubuntu 22.04.5 LTS with the 6.10.9v of the Linux kernel. We modify the AF_XDP device driver and the AF_XDP user space library of the kernel as described in Section 3 to implement Toasty. We use Linux's perf tool [8] and Intel's PCM [13] to measure cache miss rates, PCIe writes hit rates, and memory bandwidth usage. To measure throughput and packet loss, the number of packets received by the AF_XDP application is counted within the application itself and then subtracted from the transmitted packet count by the load generator, respectively. We operate AF_XDP in zero-copy and busy poll mode for all AF_XDP configurations described in the paper (Default, Ideal, and Toasty). It is important to note that we co-locate both the AF_XDP application processing packets and the AF_XDP driver responsible for moving packet buffers between the NIC and the application on the same CPU core.

We evaluate ShRing on the same server, using shring-DPDK [21] over an NVIDIA ConnectX-6 100Gbps NIC, with load generated by PktGen running on the client machine. For experiments involving multicore applications, we use 64B packets, as the 100 Gbps network link saturates for larger packet sizes. For all single core experiments, we use 512B packets unless mentioned otherwise.

4.1 Toasty vs. baseline AF_XDP

In this section, we compare the performance of several applications running over Toasty with that of different baseline configurations of AF_XDP.

Figure 7 shows the single-core saturation throughput for all applications. For the Toasty and AF_XDP baseline configurations, we refer to them as *default* and *ideal*, respectively, as described in §2. We see from the figure Toasty achieves up to 78% higher throughput (for the IDS application) than the default AF_XDP configuration, and almost matches the maximum throughput achieved by the ideal configuration. These performance gains come from better reuse of packet

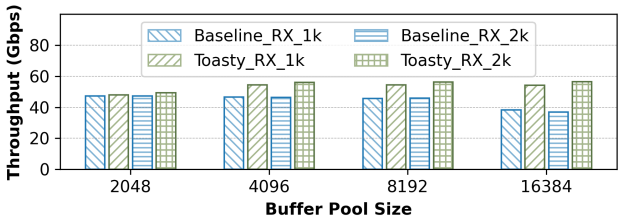


Figure 8. Toasty vs. baseline AF_XDP with varying UMEM

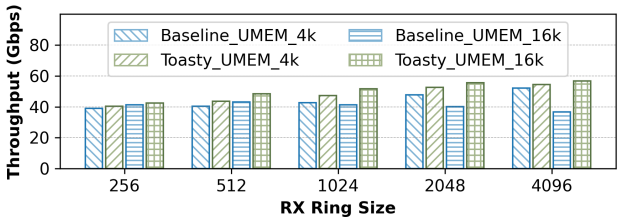


Figure 9. Toasty vs. baseline AF_XDP with varying RX ring

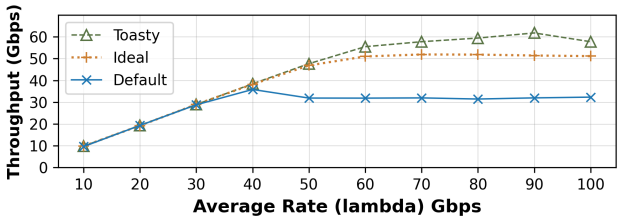


Figure 10. Throughput with Poisson traffic

I/O buffers in the CPU caches, as seen from the reduced L2 and LLC miss rates of applications with Toasty shown in Table 4 (see Table 1 for corresponding results of default and ideal configurations). We clearly see that the applications experience near-zero LLC misses with Toasty, much like with the optimally chosen static buffer pool size of the ideal configuration, because Toasty adaptively converges to the optimal buffer pool size. Improved cache performance over the default AF_XDP configuration eventually translates to the application fetching packet data from the cache instead of the main memory, leading to fewer memory stalls and improved Instruction Per Cycle (IPC), as seen in Table 5.

We would like to point out that throughput gains with Toasty vary across applications, due to the specific characteristics of the application. For example, the L2Fwd application has a more modest throughput gain because there is a longer delay to recycle packets when transmission is involved. Although the kernel changes in Toasty expedite the recycling of transmitted buffers, this change also imposes an overhead due to the reduced TX batch size, resulting in an overall lower throughput gain. Similarly, the decryption NF is highly compute-intensive and does not stress the network I/O enough to create cache pressure, which again translates to lower throughput gains. However, even for applications showing modest throughput gains, we still see a significant improvement in tail latency of the applications. Table 6 shows latency for the L2Fwd application with Toasty as compared to the baselines. We clearly see that Toasty does a lot better in terms of tail latencies (36.2% gain for 99 percentile and 2.6× gain for 99.99 percentile) when compared with the L2Fwd application running over the default configuration, in spite of Toasty delivering modest throughput gains. These results show that Toasty provides performance benefits to the different types of applications, along different dimensions of performance.

Next, we compare Toasty with a more extensive set of baseline AF_XDP configurations, besides the "default" and "ideal" ones. Figures 8 and 9 show the throughput of the NAT application, with Toasty and baseline AF_XDP, where we vary the packet buffer pool (UMEM) size and RX ring size in many different ways. We see that Toasty outperforms the baseline across various configurations. The performance gains of Toasty over the baseline are smaller at very low values of the buffer pool and RX ring sizes, because the limited availability of buffers in these configurations leaves little to no room for Toasty to improve throughput. However, Toasty outperforms the baseline AF_XDP at all reasonable configurations, e.g., RX ring size 1K to 4K, and UMEM size of 4K to 16K. Furthermore, the performance of Toasty is less sensitive to these configuration parameters themselves because Toasty dynamically adapts the buffer pool size. This implies that Toasty eliminates the need for fine-tuning various parameters, such as UMEM size or RX ring size. One can simply set these parameters to some reasonable defaults and trust Toasty to converge to the optimal values.

4.2 Variable traffic conditions

We now compare the performance of Toasty with the baseline AF_XDP under different types of bursty traffic patterns. First, we consider the microburst and Poisson traffic workloads described in §2. Figure 10 shows the throughput of the NAT application when processing Poisson distributed traffic, as we vary the average of the Poisson distribution from 10 to 100 Gbps. We see that the application throughput flattens at different levels for Toasty and the baselines, and the maximum throughput achieved by Toasty exceeds that of the baselines. The results with the other applications were similar. Note that the throughput of Toasty is higher than even the "ideal" configuration, because while the ideal configuration uses a smaller size of packet buffer pool that gives the highest throughput under steady traffic, it can suffer from packet losses under bursty traffic. On the other hand, Toasty can adapt the buffer pool size to traffic bursts better, leading to lower packet drops and overall higher throughput. To verify this claim, Table 7 shows the packet drop percentage with Toasty for all applications, with the microburst and Poisson traffic patterns (the corresponding ideal and default AF_XDP packet drop % are shown in Tables 2 and 3 of §2). We see from these results that Toasty handles bursts better than ideal AF_XDP, and behaves almost similar to the default configuration of AF_XDP that uses a large packet buffer pool.

Our throughput measurements so far allowed for a very small packet drop rate of 0.1%, that transport protocols can recover from. Now, we compute the No-Drop Rate (NDR), i.e., maximum throughput that can be sustained without any packet loss [26]. Figure 11 shows the NDR for all applications, with Toasty and the baseline AF_XDP configurations. We see from the figure that while the throughput values themselves

Traffic	Toasty Packet Drop %					
	NAT	IDS	Dec.	L2Fwd	MICA	Maglev
Microburst	0.01	0.02	0	0.02	1.68	0.02
Poisson	0.482	0.156	1.17	0.029	1.16	0.303

Table 7. Packet drop (%) with varying traffic conditions

Burst Size	Mode	Packet Drop %			
		Freq 2	Freq 5	Freq 10	Freq 20
1024	Default	0.00	0.00	0.030	0.035
	Ideal	0.080	0.080	0.088	0.087
	Toasty	0.010	0.010	0.040	0.050
2048	Default	0.00	0.00	0.050	0.073
	Ideal	11.854	26.472	27.039	31.167
	Toasty	0.010	0.020	0.030	0.640
4096	Default	0.00	0.00	0.070	0.082
	Ideal	20.546	34.867	35.562	41.681
	Toasty	1.43	2.31	3.31	5.752

Table 8. Packet drop % with varying burst size and frequency

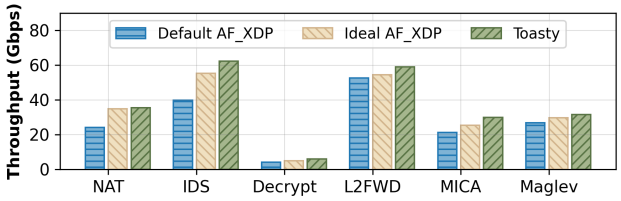


Figure 11. No-drop rate: Toasty vs. baselines

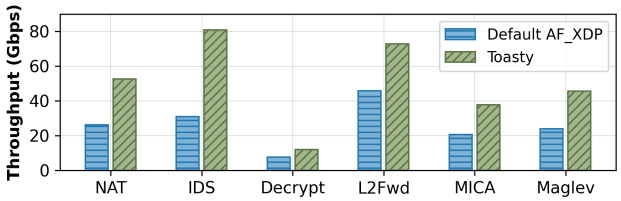


Figure 12. Toasty vs. baseline with co-located workloads

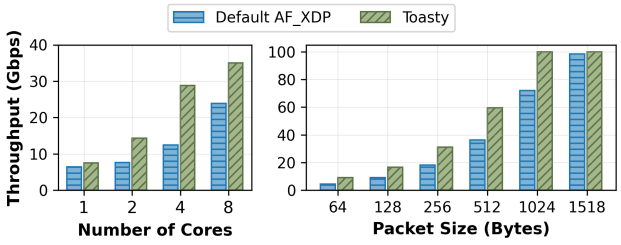


Figure 13. Toasty with varying packet size and CPU cores

are slightly lower, the relative trends between Toasty and the baselines remain the same.

Finally, we evaluate Toasty by varying the microburst size and frequency. Recall that our experiments in §2 considered one microburst of 2048 512B packets every 1ms. We now vary the burst size from 1024 to 4096 packets of 512B each. We vary the burst frequency by sending K bursts every 5ms, where K varies from 2 to 20. Table 8 compares the packet drop percentage of Toasty with that of the default and ideal AF_XDP baselines, for the NAT application. The results show that packet drop rates increase with both larger burst sizes and higher burst frequencies across all configurations. Toasty exhibits packet drop behavior comparable to default AF_XDP, while the "ideal" configuration with a small packet buffer pool and RX ring size sees significant packet drops.

4.3 Co-located workloads

We now evaluate the performance of Toasty when running under the realistic scenario of memory pressure from co-located memory-intensive applications. We simulate memory pressure using Intel's MLC tool [12] running on 8 cores and generating memory read/write requests. Figure 12 shows the single-core saturation throughput of all applications, for Toasty and the default AF_XDP baseline. We see from the figure that Toasty is able to adjust to this memory pressure much better than the baseline by reusing a smaller set of memory buffers in cache, leading to 30-86% performance gains over the default baseline.

4.4 Varying packet size and application cores

We now evaluate the scalability of Toasty with varying number of application cores and for different packet sizes. Our setup consists of a replicated application running on multiple cores, each of which is associated with a separate NIC RX ring. The NIC distributes the packets to these RX rings using Receive Side Scaling (RSS) [9]. We also tag separate buffer pools to every RX ring, and hence the Toasty driver still moves a particular buffer between only one FQ and one NIC RX ring. We observe that the performance of Toasty scales almost linearly for all applications as the number of cores increases, until the network saturates. Figure 13 shows saturation throughput of the NAT application on both Toasty and default AF_XDP, as we vary the number of cores from 1 to 8 (for 64B packets), and the packet size from 64B to 1518B (for single core). We did not use larger packet sizes for the multicore experiments, as the application was able to saturate the 100 Gbps link using just 2 cores for Toasty, and using 4 cores for default AF_XDP with 512B packets. The figure shows that the throughput of Toasty scales nearly linearly with the increasing number of cores for 64B packets, and Toasty is able to scale better to higher packet rates as compared to the default AF_XDP.

4.5 Comparison with related work

In this section, we compare Toasty with several state-of-the-art systems designed to solve similar problems.

DPDK LIFO mempool [25] is a buffer pool configuration in DPDK, in which the packet buffer pool is maintained as a LIFO stack instead of a FIFO queue. We now compare Toasty with this LIFO-based packet buffer pool implementation in DPDK, and also with the user-space only LIFO buffer pool implementation of Toasty (termed User Toasty in this section). All experiments in this section have been configured to run with the same buffer pool size (16384 buffers) and the same

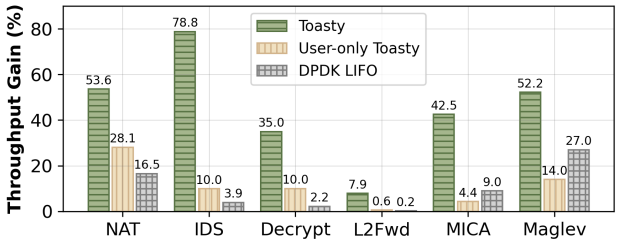


Figure 14. Toasty vs. User Toasty vs. DPDK LIFO mempool

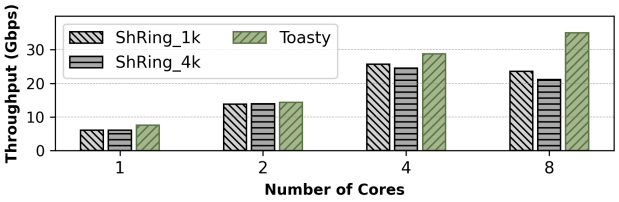


Figure 15. Toasty vs. ShRing

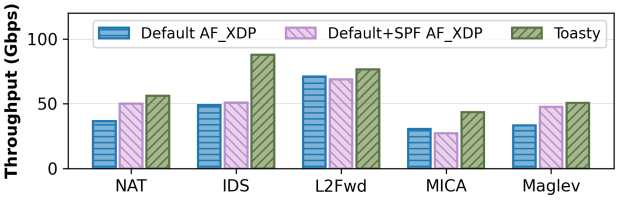


Figure 16. Toasty vs. software prefetching

NIC RX ring size (4096 buffers). Figure 14 shows throughput gains for Toasty, User Toasty, and DPDK LIFO over their corresponding default configurations (the default AF_XDP configuration or the default DPDK FIFO-based mempool configuration, as the case may be). We see that the gains obtained with User Toasty over the default configuration are comparable to what the DPDK LIFO system is able to get over the default DPDK baseline, albeit with slightly higher cache miss rates in DPDK as compared to Toasty due to a more complicated LIFO stack implementation in DPDK. These results confirm the need for the adaptive RX ring refill logic employed by the kernel driver in Toasty in realizing the full benefits of the system.

ShRing [23] ShRing uses primitives available on state-of-the-art smart NICs to efficiently share a single RX ring across multiple cores, effectively reducing the working set size of a multi-core network application, resulting in better cache hit rates. Figure 15 shows the saturation throughput when the network application (NAT in this case) runs on different numbers of cores (from 1 to 8). The throughput of ShRing is shown for two RX ring sizes (1024 and 4096 buffers), which are shared among all cores, while each core of Toasty has an RX ring of size 4096 that is adaptively refilled. We can see that Toasty performs better than ShRing, especially at a higher number of cores, while also working on simpler commodity NIC hardware. In our experimental setup, ShRing achieves its best performance with 4 cores, contrary to the

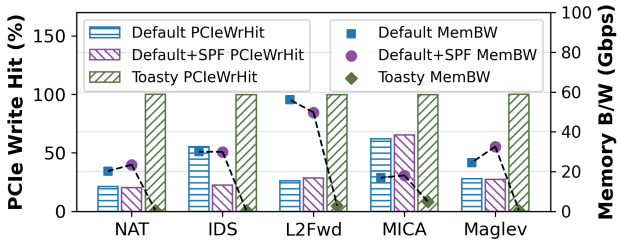


Figure 17. PCIe write hit rate and memory BW with SPF

8-core scalability mentioned in the original paper [23]. We further observe that ShRing requires 1 GB hugepages to achieve good performance, whereas Toasty does not impose such constraints. Next, we measure the packet drop rates of ShRing and Toasty with bursty traffic (Poisson distributed with average rate 30 Gbps, as described in §2). We find that the packet drop rates of ShRing (1024 ring size), ShRing (4096 ring size), and Toasty are 0.53%, 1.18%, and 0.01% respectively. These results show that the adaptively refilled RX rings in Toasty are more resilient to traffic bursts than the shared RX ring in ShRing, even while delivering comparable or better throughput due to better cache utilization.

Software prefetching [2] improves memory access performance by loading data into the processor's cache before it is actually needed, using nonblocking prefetch instructions. It can be used to bring packet buffers into the cache before the application goes on to process them, partly hiding the leaky DMA problem that occurs due to large packet buffer pools. In this section, we investigate whether software prefetching (SPF) can do as well as Toasty in mitigating the leaky DMA problem. For our SPF baseline, the applications use SPF to prefetch packets in the RXQ. Figure 16 shows the throughput of the default AF_XDP configuration with and without SPF, and the throughput with Toasty. The results show that SPF does move the needle a bit and improves the throughput of some applications running over the default AF_XDP configuration to a value much closer to Toasty, e.g., for NAT and Maglev. However, this gain in performance comes at a cost. Figure 17 shows the PCIe write hit rate and DRAM bandwidth used in these experiments. The results clearly show that applications running over SPF consume a higher memory bandwidth when compared with Toasty (as much as 35× for Maglev and 25× for NAT, the two apps which are closest to Toasty in terms of throughput), in order to proactively fetch packet buffers evicted from LLC. This shows that applications running with SPF essentially use extra resources (memory bandwidth) to increase their LLC hit rates and improve throughput when compared to Toasty, which delivers more throughput using a fraction of the resources.

4.6 Microbenchmarks

Swap Overheads. Toasty requires one swap operation per processed packet while replenishing the FQ buffer. Here, we quantify the computational overhead of swap operations

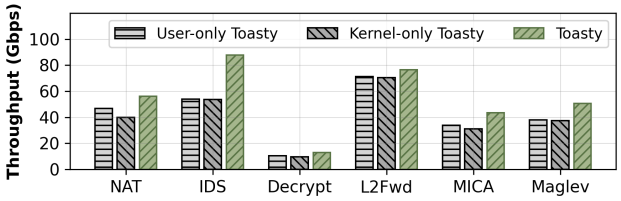


Figure 18. User-space only and kernel-space only variants

Modes	L2 Miss % (LLC Miss %)					
	NAT	IDS	Dec.	L2Fwd	MICA	Mag.
User only	17.85 (0.085)	54.26 (0.05)	50.25 (0.49)	20.41 (0.49)	28.96 (0.50)	18.00 (0.14)
Kernel only	33.24 (1.44)	63.23 (9.10)	65.45 (19.06)	35.27 (1.23)	35.18 (5.15)	32.09 (1.48)

Table 9. L2 and LLC cache miss rates for user-space only and kernel-space only Toasty variants

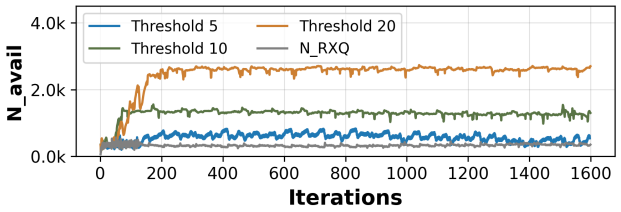


Figure 19. Number of available buffers vs. k

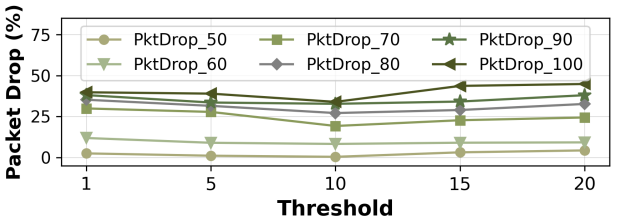


Figure 20. Performance of Toasty for different values of k

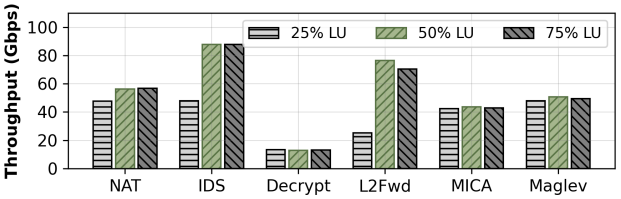


Figure 21. Sensitivity to link util (LU) threshold

introduced by Toasty. To isolate the cost, we executed 1M swap operations and measured a cumulative execution time of 4.9ms. For comparison, our NAT implementation achieves a throughput of 52.9 Gbps, which implies a packet processing time of approximately 72ms for 1M packets (512B). Hence, the swap overheads account for 4.9ms of the total processing time of 72ms, which results in an overhead of 6.8% that is more than offset by the performance gains of Toasty.

Ablation study. We now quantify the performance gains of the individual components of Toasty, namely, the user-space LIFO buffer pool, and the adaptive RX ring refill in the kernel driver. Figure 18 shows the throughput of three different variants of Toasty, one with only the userspace LIFO buffer pool, one with only the adaptive RX refill in the kernel driver, and the final complete system with both components. We see from the figure that the components of Toasty do not realize full performance gains in isolation, but work synergistically to improve performance in the complete system. With the user-space LIFO buffer pool enabled in isolation, warm buffers are reused at the application level; however, the NIC RX ring still behaves as a FIFO queue, allowing cold buffers to accumulate at the head of the ring and limiting the cache locality benefits. With the kernel-space adaptive RX ring enabled in isolation, the RX ring dynamically adjusts to traffic load, but cannot recycle warm buffers, resulting in continued cache misses. These observations are further verified in the Table 9, where we see that the L2 and LLC cache misses are higher in the case of kernel-space only Toasty as compared to the user-space only Toasty. However, the cache miss rate of the kernel-only Toasty is still slightly lower than that of default AF_XDP (shown in Table 1), which shows the effectiveness of our adaptive RX refill mechanism.

Sensitivity to the threshold parameter k. Now, we test the sensitivity of Toasty to the choice of the threshold parameter k in our design, which is the multiplier factor relative to the incoming packet rate that is used to compute the number of free buffers the kernel driver fills in the hardware RX ring. First, Figure 19 shows how the number of available buffers in the RX ring varies for different values of k, for the IDS application. We see from the figure that higher values of k lead to the driver maintaining a larger number of free buffers in the RX ring, which increases resilience to packet drops but also increases the I/O working set size and hurts performance. On the other side, with a lower value of k, Toasty maintains a very small number of buffers, leading to more packet drops. Next, Figure 20 shows the packet drop rates of the NAT application under varying average rates of Poisson traffic. We see from the figure that a value of $k = 10$ leads to the sweet spot in performance, striking a balance between throughput and packet drops. At this value of k, Toasty is able to maintain a low working set size that fits in LLC DDIO ways under steady traffic conditions, while being able to quickly adapt to bursty traffic without dropping packets, justifying the choice of the threshold parameter in our system. Of course, slightly different threshold values also give comparable performance, and our design is not too sensitive to the choice of the threshold.

Sensitivity to the link utilization threshold. Our design uses a threshold of 50% to decide if a link is reasonably utilized or not. Figure 21 shows the throughput of Toasty for different values of this threshold. We see that a threshold of 25% is too low to classify a link as heavily utilized and start filling the RX ring aggressively. The threshold of 50% strikes a good balance; it is high enough to filter out normal

traffic fluctuations while still enabling timely detection and response to bursty or high-rate traffic. A threshold of 75% tends to delay the system's reaction to some traffic bursts, but is not too much worse than the threshold of 50% that is currently used in our design. We conclude that our system is fairly stable for a range of threshold values around the chosen threshold value.

5 Related Work

We now summarize related work that has identified DDIO performance issues and proposed solutions for the same.

Leaky DMA problem. ResQ [30] was one of the first works that discussed the leaky DMA problem and proposed that the RX ring should be sized to fit within the LLC to avoid this problem. Our experiments clearly demonstrate the pitfalls of this approach. ShRing [23] questions the wisdom of using a separate NIC RX ring for every CPU core, and argues that using multiple RX rings increases the I/O working set size, leading to the leaky DMA problem.

Unnecessary writebacks. IDIO [1] and Sweeper [31] suggest microarchitectural changes to reduce unnecessary write-back traffic between L2 and L3, and between L3 and memory respectively, caused by network I/O workloads in the context of DDIO. Both solutions identify that packet buffers, once used up by the application, hold no value until new data is DMA'd into them by the NIC, and hence, propose hardware solutions (e.g., invalidating processed I/O buffers automatically, or via explicit hardware instructions), to avoid writing them back from a mid-level cache to LLC or memory. Toasty proposes a software-only solution to this problem.

Fine-tuning DDIO. Farshin et al. [6] were able to identify that Intel's DDIO DMA'd packets into the last two LLC ways, and hence is only limited to using a small portion of the LLC. They further found ways to tune DDIO by controlling the LLC ways used by DDIO, increasing network I/O capacity. CacheDirector [5] optimized DDIO by DMA'ing incoming packets into the LLC slice closest to the core, which would eventually process that particular packet. Wang et.al. [32] reverse-engineered DDIO and found that when a packet buffer resident in L2 was forwarded back to the NIC, it was first moved to two separate ways in LLC (other than the DDIO ways), before the NIC could read these buffers, showing that network I/O could be using more cache space in LLC than anticipated in the case of DDIO. IAT [34] builds an LLC management mechanism that dynamically adjusts the number of LLC ways for DDIO, for the tenants that demand more LLC capacity. In addition, IAT dynamically chooses the tenants that share LLC resources with DDIO to minimize the performance interference. Toasty is orthogonal to research that fine-tunes DDIO.

Redesigning the NIC-CPU interface. Enso [28] argues that a high percentage of the PCIe interface is wasted in transferring metadata in the current NIC-CPU interface, and advocates for a simpler interface that places all incoming packets contiguously in a single large buffer. This also results in a better fit in cache as a side effect, in the context of DDIO. Pismenny et al. [22] identify that most networking applications only work on the header of the network packet. Hence, they advocate using the available on-NIC memory to store payloads, and only DMA packet headers to be processed by the application. ShRing [23] leverages features available in modern smart NICs, and proposes sharing a common RX ring across multiple cores to reduce the I/O working set size. To avoid the problem of this common ring being shared unfairly across cores due to load imbalance, RxBisect[24] proposes dividing the RX ring into two separate rings, an allocation ring and a bisected reception ring corresponding to each producer-consumer (core-NIC) pair. CEIO [17] is a cache-efficient hardware architecture for the NIC-CPU datapath, which utilizes a proactive credit-based I/O rate manager in the NIC to minimize LLC contention. The above solutions either require a redesigned hardware NIC-CPU interface or exploit certain capabilities made available in some state-of-the-art smart NICs. In contrast, Toasty does not depend on any new hardware capabilities and works with commodity NICs.

DPDK stack mempool. DPDK has the option of structuring the buffer pool as a stack [25], as an alternative to the traditional FIFO ring-based mempool. The LIFO stack mempool was introduced to improve cache locality, allowing applications to reuse recently freed, cache-warm buffers more efficiently. However, this paper argues that such changes alone are not enough to mitigate the leaky DMA problem, and also shows results (§4.5) supporting this claim.

6 Conclusion

This paper presented Toasty, a software framework built atop the AF_XDP kernel bypass framework, that avoids the trade-off between high network I/O performance and resilience to packet drops that arises when sizing the network packet buffer pool. Toasty structures the packet buffer pool as a LIFO stack instead of a FIFO queue, and adapts the number of free buffers populated into the hardware RX ring based on traffic conditions. Together, these changes ensure that Toasty reuses a small working set of cache-warm buffers in steady traffic conditions, while also pulling in a larger number of packet buffers during traffic bursts to avoid packet drops. Toasty can be deployed on commodity NIC hardware and delivers impressive performance gains over the default AF_XDP configuration across various network applications and traffic conditions. While recent research has leveraged new features in modern smart NICs or explored ways to redesign the NIC-CPU interface to optimize network I/O, Toasty explores a different point in the design space and shows that one can reap significant performance gains with software changes on existing hardware. We will open source the code of Toasty upon acceptance of our publication.

References

[1] Mohammad Alian, Siddharth Agarwal, Jongmin Shin, Neel Patel, Yifan Yuan, Daehoon Kim, Ren Wang, and Nam Sung Kim. 2022. IDIO: Network-Driven, Inbound Network Data Orchestration on Server Processors. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 480–493.

[2] David Callahan, Ken Kennedy, and Allan Porterfield. 1991. Software prefetching. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Santa Clara, California, USA) *(ASPLOS IV)*. Association for Computing Machinery, New York, NY, USA, 40–52.

[3] Cilium. 2024. AF XDP. https://docs.ebpf.io/linux/concepts/af_xdp/.

[4] Danielle E. Eisenbud, Cheng Yi, Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Cilingiroglu, Bin Cheyney, Wentao Shang, and Jinnah Dylan Hosein. 2016. Maglev: a fast and reliable software network load balancer. In *Proceedings of the 13th Usenix Conference on Networked Systems Design and Implementation* (Santa Clara, CA) *(NSDI'16)*. USENIX Association, USA, 523–535.

[5] Alireza Farshin, Amir Roozbeh, Gerald Q. Maguire, and Dejan Kostić. 2019. Make the Most out of Last Level Cache in Intel Processors. In *Proceedings of the Fourteenth EuroSys Conference 2019* (Dresden, Germany) *(EuroSys '19)*. Association for Computing Machinery, New York, NY, USA, Article 8, 17 pages.

[6] Alireza Farshin, Amir Roozbeh, Gerald Q. Maguire, and Dejan Kostić. 2020. Reexamining direct cache access to optimize I/O intensive applications for multi-hundred-gigabit networks. In *Proceedings of the 2020 USENIX Conference on Usenix Annual Technical Conference (USENIX ATC'20)*. USENIX Association, USA, Article 46, 17 pages.

[7] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. 2020. Caladan: mitigating interference at microsecond timescales. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 16, 17 pages.

[8] Brendan Gregg. 2024. perf Examples. https://www.brendangregg.com/perf.html.

[9] Tom Herbert and Willem de Bruijn. 2024. Scaling in the Linux Networking Stack. https://www.kernel.org/doc/Documentation/networking/scaling.txt.

[10] Intel. 2024. Data Plane Development Kit. https://www.dpdk.org/.

[11] Intel. 2024. Intel® Data Direct I/O Technology. https://www.intel.com/content/www/us/en/io/data-direct-i-o-technology.html.

[12] Intel. 2024. Intel® Memory Latency Checker. https://www.intel.com/content/www/us/en/developer/articles/tool/intelr-memory-latency-checker.html.

[13] Intel. 2024. Intel® Performance Counter Monitor. https://github.com/intel/pcm.

[14] Kostis Kaffes, Timothy Chong, Jack Tigar Humphries, Adam Belay, David Mazières, and Christos Kozyrakis. 2019. Shinjuku: preemptive scheduling for μsecond-scale tail latency. In *Proceedings of the 16th USENIX Conference on Networked Systems Design and Implementation* (Boston, MA, USA) *(NSDI'19)*. USENIX Association, USA, 345–359.

[15] Magnus Karlsson and Bjorn Topel. 2018. The Path to DPDK Speeds for AF XDP. In *Linux Plumbers Conference, Vancouver*.

[16] Hyeontaek Lim, Dongsu Han, David G. Andersen, and Michael Kaminsky. 2014. MICA: a holistic approach to fast in-memory key-value storage. In *Proceedings of the 11th USENIX Conference on Networked Systems Design and Implementation* (Seattle, WA) *(NSDI'14)*. USENIX Association, USA, 429–444.

[17] Bowen Liu, Xinyang Huang, Qijing Li, Zhuobin Huang, Yijun Sun, Wenxue Li, Junxue Zhang, Ping Yin, and Kai Chen. 2025. CEIO: A Cache-Efficient Network I/O Architecture for NIC-CPU Data Paths. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) *(SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 381–394.

[18] Amy Ousterhout, Joshua Fried, Jonathan Behrens, Adam Belay, and Hari Balakrishnan. 2019. Shenango: achieving high CPU efficiency for latency-sensitive datacenter workloads. In *Proceedings of the 16th USENIX Conference on Networked Systems Design and Implementation* (Boston, MA, USA) *(NSDI'19)*. USENIX Association, USA, 361–377.

[19] Federico Parola, Roberto Procopio, Roberto Querio, and Fulvio Risso. 2023. Comparing User Space and In-Kernel Packet Processing for Edge Data Centers. *SIGCOMM Comput. Commun. Rev.* 53, 1 (April 2023), 14–29.

[20] Federico Parola, Shixiong Qi, Anvaya B. Narappa, K. K. Ramakrishnan, and Fulvio Risso. 2024. SURE: Secure Unikernels Make Serverless Computing Rapid and Efficient *(SoCC '24)*. Association for Computing Machinery, New York, NY, USA, 668–688.

[21] Boris Pismenny. 2024. ShRing-DPDK. https://github.com/BorisPis/shRing-dpdk.

[22] Boris Pismenny, Liran Liss, Adam Morrison, and Dan Tsafrir. 2022. The benefits of general-purpose on-NIC memory. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 1130–1147.

[23] Boris Pismenny, Adam Morrison, and Dan Tsafrir. 2023. ShRing: Networking with Shared Receive Rings. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 949–968. https://www.usenix.org/conference/osdi23/presentation/pismenny

[24] Boris Pismenny, Adam Morrison, and Dan Tsafrir. 2025. Disentangling the dual role of NIC receive rings. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (Boston, MA, USA) *(OSDI '25)*. USENIX Association, USA, Article 36, 19 pages.

[25] DPDK Project. [n. d.]. Stack Mempool Driver. https://doc.dpdk.org/guides/mempool/stack.html. Accessed: 2025-04.

[26] RFC2544. [n. d.]. Benchmarking Methodology for Network Interconnect Devices. https://www.rfc-editor.org/rfc/rfc2544.txt. Accessed: 2025-12.

[27] Luigi Rizzo. 2012. Netmap: a novel framework for fast packet I/O. In *Proceedings of the 2012 USENIX Conference on Annual Technical Conference* (Boston, MA) *(USENIX ATC'12)*. USENIX Association, USA, 9.

[28] Hugo Sadok, Nirav Atre, Zhipeng Zhao, Daniel S. Berger, James C. Hoe, Aurojit Panda, Justine Sherry, and Ren Wang. 2023. Enso: A Streaming Interface for NIC-Application Communication. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 1005–1025. https://www.usenix.org/conference/osdi23/presentation/sadok

[29] Dan Tang, Yungang Bao, Weiwu Hu, and Mingyu Chen. 2010. DMA cache: Using on-chip storage to architecturally separate I/O data from CPU data for improving I/O performance. In *HPCA - 16 2010 The Sixteenth International Symposium on High-Performance Computer Architecture*. 1–12.

[30] Amin Tootoonchian, Aurojit Panda, Chang Lan, Melvin Walls, Katerina Argyraki, Sylvia Ratnasamy, and Scott Shenker. 2018. ResQ: enabling SLOs in network function virtualization. In *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation* (Renton, WA, USA) *(NSDI'18)*. USENIX Association, USA, 283–297.

[31] Marina Vemmou, Albert Cho, and Alexandros Daglis. 2023. Patching up Network Data Leaks with Sweeper. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture* (Chicago, Illinois, USA) *(MICRO '22)*. IEEE Press, 464–479.

[32] Minhu Wang, Mingwei Xu, and Jianping Wu. 2022. Understanding I/O Direct Cache Access Performance for End Host Networking. *Proc. ACM Meas. Anal. Comput. Syst.* 6, 1, Article 22 (Feb. 2022), 37 pages.

[33] Keith Wiles. 2024. DPDK PktGen. https://github.com/pktgen/Pktgen-DPDK.

[34] Yifan Yuan, Mohammad Alian, Yipeng Wang, Ren Wang, Ilia Kurakin, Charlie Tai, and Nam Sung Kim. 2021. Don't forget the I/O when allocating your LLC. In *Proceedings of the 48th Annual International Symposium on Computer Architecture* (Virtual Event, Spain) *(ISCA '21)*. IEEE Press, 112–125.